

Technical Report: VOIDX — OSINT & Security Scanner

Subject: Python-based command-line OSINT and port-scanning utility **Author (per README):** FSociety 353

Report type: Technical / functional documentation

1. Overview

VOIDX is a single-file Python command-line tool that combines open-source intelligence (OSINT) gathering with basic network port scanning. It presents the user with a simple interactive menu and, given a target IP address or domain, runs a sequence of information-gathering and scanning routines against it.

The tool is built entirely on top of well-known third-party and standard libraries — it does not implement any novel scanning or exploitation logic of its own. It is structured as a single script (`voidx.py`) with no external configuration files, driven entirely by console input.

2. Dependencies

Library	Purpose in this tool
<code>socket</code> (standard library)	DNS resolution and raw TCP port connection attempts
<code>threading</code> (standard library)	Runs the port scan across multiple ports concurrently
<code>requests</code>	HTTP calls to the geolocation API and to the target's web server for header retrieval
<code>whois</code>	Performs WHOIS domain registration lookups
<code>platform</code> (standard library)	Reads information about the <i>local</i> machine running the script
<code>scapy</code>	Constructs and sends raw ARP packets to resolve a MAC address
<code>sys</code> , <code>json</code> , <code>time</code> (standard library)	Argument handling, JSON formatting, and (imported but unused) timing

None of these are exploitation frameworks; they are general-purpose networking, data-formatting, and OSINT libraries commonly used in scripting and systems programming.

3. Program Structure

The script is organized into seven functions plus a `main()` entry point that drives a `while True` interactive menu loop.

3.1 `print_banner()`

Prints an ASCII-art “VOIDX” banner and a short tagline when the program starts. Purely cosmetic — no functional effect.

3.2 `get_mac(ip)`

Uses `scapy` to broadcast an ARP request for the given IP address and reads the MAC address from the reply. This only works for hosts on the same local network segment (ARP does not route across networks) and requires root privileges because raw packet crafting needs elevated access on most operating systems.

3.3 `get_attack_vector(port)`

A static lookup table mapping nine common port numbers (21, 22, 23, 25, 53, 80, 443, 3306, 3389) to a short descriptive string of attack categories historically associated with services on that port (e.g., port 22 → “SSH Bruteforce, Credential Stuffing”). This is a labeling function only — it does not perform or automate any of the listed attacks. Any port not in the table returns “Unknown attack vector”.

3.4 `gather_osint(target)`

The core information-gathering routine, run in four steps:

1. **WHOIS lookup** — calls `whois.whois(target)` and prints domain name, registrar, creation year, expiration date, name servers, status, listed emails, organization, and country, wrapped in a try/except that prints a fallback message if the lookup fails (e.g., for a bare IP or a domain with no WHOIS record).
2. **DNS resolution** — resolves the target to an IP address via `socket.gethostbyname()`. If this fails, the function returns early, since no IP means the rest of the reconnaissance can't proceed.
3. **Geolocation lookup** — sends a request to the free `ip-api.com` HTTP API and prints the returned JSON (city, region, ISP, coordinates, etc.) if the lookup succeeds; prints a "private IP" message otherwise.
4. **Local device info** — prints the OS, release version, machine architecture, and processor of the machine running the script (not the target). It also attempts a MAC address lookup via `get_mac()`, but only if running on Linux and `sys.argv[0]` starts with the literal string `"sudo"` — in practice this check rarely evaluates true even when run with `sudo`, because `sudo` does not typically alter `sys.argv[0]`, so this branch is effectively dead code in most environments.

3.5 `http_scan(target)`

Sends a single unauthenticated GET request to `http://{target}` and prints the response headers as JSON. Wrapped in a blanket try/except that prints a generic failure message on any error (timeout, connection refused, DNS failure, etc.).

3.6 `scan_port(target, port)`

Opens a raw TCP socket and calls `connect_ex()` against a single port with a 1-second timeout. A return value of 0 is treated as "open," anything else as "closed or filtered." If a port is open, it looks up and prints the corresponding entry from `get_attack_vector()`. This is a plain TCP connect scan (the same fundamental technique `nmap -sT` uses) — no SYN stealth scanning, service/version fingerprinting, or banner grabbing is performed here (banner data is separately obtained by `http_scan` for web ports only).

3.7 `main()`

Presents the interactive loop:

- Option 1 — take a raw IP address as the target.
- Option 2 — take a domain name, strip any `http://` / `https://` prefix and path, validate that it contains a `.`, then resolve it to an IP via `socket.gethostbyname()`.
- Option 3 — a placeholder that prints a "coming soon" message referencing planned social engineering and brute-force features (see §6) and loops back to the menu without running anything.
- Option 4 — exits the program.

For options 1 and 2, once a target is established, the script always runs `gather_osint()` and `http_scan()`, then prompts for a start and end port number and spawns one Python thread per port in that range to run `scan_port()` concurrently, joining all threads before printing a completion message.

4. Walkthrough Notes

A few points worth calling out explicitly, since they're easy to misread from the code alone:

- **The menu loop drives everything.** `main()` is a `while True` loop that keeps re-displaying the menu until option 4 is chosen. Options 1 and 2 both end by producing a `target` variable holding an IP address — option 1 takes it directly from the user; option 2 takes a domain, strips any `http://` / `https://` prefix and anything after the first `/`, checks that a `.` is present, then resolves it to an IP via `socket.gethostbyname()`. Once `target` is set, the script unconditionally runs OSINT gathering, then an HTTP header grab, then asks for a port range and scans it — there's no way to run just one of these steps.
- **WHOIS only makes sense for domains, not bare IPs.** `whois.whois()` queries public WHOIS servers for registration data — registrar, creation/expiry dates, name servers, listed abuse/admin emails, and so on. Feeding it a raw IP (or an unregistered-looking string) doesn't crash the script; the bare `except:` catches the failure and prints "WHOIS information not found" instead.

- **DNS resolution is a hard gate.** If `socket.gethostbyname()` fails, `gather_osint()` returns immediately rather than continuing — there’s no IP to hand to geolocation or the rest of the pipeline.
- **Geolocation is a plain, unauthenticated HTTP call.** It’s a GET to `ip-api.com`’s free JSON endpoint — no API key, no HTTPS, no rate-limit handling. Private/reserved-range IPs make that API return a `"fail"` status, which the script interprets and reports as “Private IP detected.”
- **The “device information” step describes the operator’s machine, not the target.** `platform.system()`, `platform.release()`, `platform.machine()`, and `platform.processor()` all report on the computer running VOIDX. This is the part of the code most likely to be misread as target fingerprinting — it isn’t; it looks like leftover debugging/support output.
- **The MAC-address gate is broken.** It tries to check for root privileges by testing `sys.argv[0].startswith("sudo")`. But running `sudo python3 voidx.py` sets `sys.argv[0]` to `"voidx.py"` or the script’s path — not the string `"sudo"` — so this condition essentially never evaluates true in normal use, and the ARP-based MAC lookup effectively never runs even when the script does have the root privileges it needs.

5. Data Flow Summary



All output is printed directly to the console; nothing is written to disk, logged to a file, or transmitted anywhere beyond the outbound requests described above (WHOIS query, one HTTP GET to the target, one HTTP GET to `ip-api.com`, and raw TCP connection attempts during the port scan).

6. Current vs. Planned Feature Set

The README advertises a broader feature list than the current script implements. Comparing the two:

README claim	Present in <code>voidx.py</code> ?
WHOIS Lookup	✓ Yes (§3.4, step 1)
DNS Lookup	✓ Yes (§3.4, step 2)
Geolocation Tracking	✓ Yes (§3.4, step 3)
Device Information Gathering	✓ Yes — but of the local machine, not the target (§3.4, step 4)
HTTP Headers Analysis	✓ Yes (§3.5)
Port Scanning	✓ Yes (§3.6)
Attack Predictions Based on Open Ports	✓ Yes, as a static 9-entry lookup table (§3.3)
Real IP Detection Behind Cloudflare	✗ Not present in this script
Website History (Archived Records)	✗ Not present in this script
Security Incidents (Check for Hacked Sites)	✗ Not present in this script
Social Engineering Attacks	✗ Listed under “Upcoming Features”; menu option 3 is a placeholder only
Brute Force Techniques	✗ Listed under “Upcoming Features”; not implemented

7. Known Limitations / Code Observations

- **Bare `except:` clauses** are used throughout (WHOIS, geolocation, HTTP scan, DNS resolution). This silences all exceptions, including ones unrelated to the expected failure mode (e.g., a typo or an unrelated runtime error would be swallowed with the same generic message), which makes the tool harder to debug or extend.

- **The sudo / MAC-address check is unreliable**, as noted in §3.4 — checking whether `sys.argv[0]` starts with `"sudo"` does not reliably detect elevated privileges on most systems, so this feature likely never activates as intended.
- **No input bounds-checking on the port range** — a user can enter an arbitrarily large range (e.g., 1-65535), which spawns one Python thread per port with no concurrency limit or connection pooling. This is functional but not resource-efficient compared to scanners that use a bounded thread pool or asynchronous I/O.
- The `time` import is unused in the current script.
- **HTTP requests are unauthenticated and unencrypted** — `http_scan()` always requests `http://`, never `https://`, and does not handle redirects, so header data on HTTPS-only sites will not be captured.
- **No output file / report generation** — despite being framed as a “scanner,” all results are printed to the console only; there is no option to export results to a file for later reference, even though the README’s installation instructions reference a `requirements.txt` file that is not included among the uploaded files.

8. Installation & Execution (per README)

The README documents a standard Linux/Termux workflow:

1. Clone the repository via `git clone .`
2. `cd` into the project folder.
3. Install dependencies with `pip install -r requirements.txt .`
4. Make the script executable with `chmod +x voidx.py .`
5. Run it with `python3 voidx.py` (or `sudo python3 voidx.py` if a Permission Denied error occurs — relevant to the ARP/MAC lookup in §3.2, which does require elevated privileges to function).

It also documents moving the tool to `~/Desktop` for convenience, and lists supported platforms as Kali Linux, Parrot OS, Ubuntu/Debian-based Linux, and Termux (Android).

9. Summary

VOIDX is a lightweight, single-file reconnaissance utility that chains together WHOIS, DNS, IP geolocation, HTTP header retrieval, and a multi-threaded TCP connect-style port scan, with a static table mapping common ports to historically associated attack categories for user reference. Functionally, it is closer to a beginner-oriented OSINT/recon script than a full scanning platform — several README-advertised features (Cloudflare bypass, site history, breach checking, social engineering, brute force) are not present in the current code and are marked as upcoming or absent entirely. The codebase is compact (roughly 130 lines) and readable, but uses broad exception handling throughout and has at least one non-functional conditional branch (the sudo check in `gather_osint()`).